VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Faculty of Computer Science and Engineering

CC02 — Lab Report

Digital Signal Processing
Lab 3

Supervisors:	Pham Cong Thai	
Students:	Nguyen Dang Duy	2252116
	Bui Thai An	2252001

Ho Chi Minh City, March 26, 2026

Contents

1 Lab 3 **2**
1.1 Overview . . . 2
1.2 Objectives . . . 2
1.3 Methodologies . . . 2
1.3.1 Hardware & Software . . . 2
1.4 Exercises . . . 2
1.4.1 Exercise 1 . . . 2
1.4.2 Exercise 2 . . . 3
1.4.3 Exercise 3 . . . 5
1.4.4 Exercise 4 . . . 7
1.4.5 Exercise 5 . . . 8
1.4.6 Exercise 6 . . . 10
1.5 Additional Exercise . . . 12
1.5.1 Exercise 2.1 . . . 12
1.5.2 Exercise 2.2 . . . 15
1.5.3 Exercise 2.4 . . . 19
1.5.4 Exercise 2.6 . . . 20

2 Reference **26**

1 Lab 3

1.1 Overview

This is the report of lab 3.

1.2 Objectives

- Review lecture notes of discrete-time signal and system
- Implement script-based manipulation functions in SciLab for discrete-time signals

1.3 Methodologies

1.3.1 Hardware & Software

- Hardware: Intel-based Laptop
- Software: Scilab (Windows OS)

1.4 Exercises

1.4.1 Exercise 1

`function [yn, yorigin] = delay (xn, xorigin, k)` performs delay operation $y(n) = x(n-k)$, where $k > 0$:

- The discrete-time signal $x(n)$ is presented by vector `xn`
- `xorigin` indicates the origin's position of signal $x(n)$.
- `yn` is vector of the output signal
- `yorigin` indicates the origin's position of signal $y(n)$.
- $x(n)$ and $y(n)$ are graphically displayed in the same figure.

For example: To determine the delay signal $y(n) = x(n–1)$ where $x(n) = \{1, -2, \underset{\uparrow}{3}, 6\}$. You can use your own function by calling `[yn, yorigin] = delay ([1, -2, 3, 6], 3, 1)`. Then, the output will be

```
yn = [1,  -2,  3,  6 ]
yorigin = 2
```

Additionally, you have to graphically display x(n) and y(n) in the same figure.

Solution

This is the source code of Exercise 1:

```
function [yn, yorigin] = delay(xn, xorigin, k)
    // 1. Calculate the new origin position
    // Shifting x(n-k) moves the origin index back by k
    yorigin = xorigin - k;
```

```
    yn = xn; // The values in the vector remain the same

    // 2. Determine time axes for plotting
    // Signal x starts at 1-xorigin and ends at length(xn)-xorigin
    nx = (1:length(xn)) - xorigin;
    ny = (1:length(yn)) - yorigin;

    // 3. Graphical Display
    clf();
    // Plot Original x(n)
    subplot(2,1,1);
    plot2d3('gnn', nx, xn);
    plot(nx, xn, 'ro');
    xtitle( Original Signal x(n) ,  n ,  Amplitude );
    xgrid();

    // Plot Delayed y(n)
    subplot(2,1,2);
    plot2d3('gnn', ny, yn);
    plot(ny, yn, 'bo');
    xtitle( Delayed Signal y(n) = x(n -  +string(k)+ ) ,  n ,  Amplitude );
    xgrid();

    printf( Original Origin: %d\n , xorigin);
    printf( New Origin (yorigin): %d\n , yorigin);
endfunction
```

The function calculates the new origin by subtracting the delay value k from the original origin (`xorigin`), effectively shifting the origin index back. The array values remain untouched (`yn = xn`). Time axes (`nx` and `ny`) are constructed by subtracting the origin index from a sequence of 1 to the signal's length. The original and delayed signals are then plotted on a shared figure using `subplot`

This is the output of Exercise 1:

1.4.2 Exercise 2

`function [yn, yorigin] = advance (xn, xorigin, k)` performs advance operation $y(n) = x(n+k)$, where $k > 0$ and:

- The discrete-time signal $x(n)$ is presented by vector `xn`.
- `xorigin` indicates the origin's position of signal $x(n)$.
- `yn` is vector of the output signal.
- `yorigin` indicates the origin's position of signal $y(n)$.
- $x(n)$ and $y(n)$ are graphically displayed in the same figure.

Solution

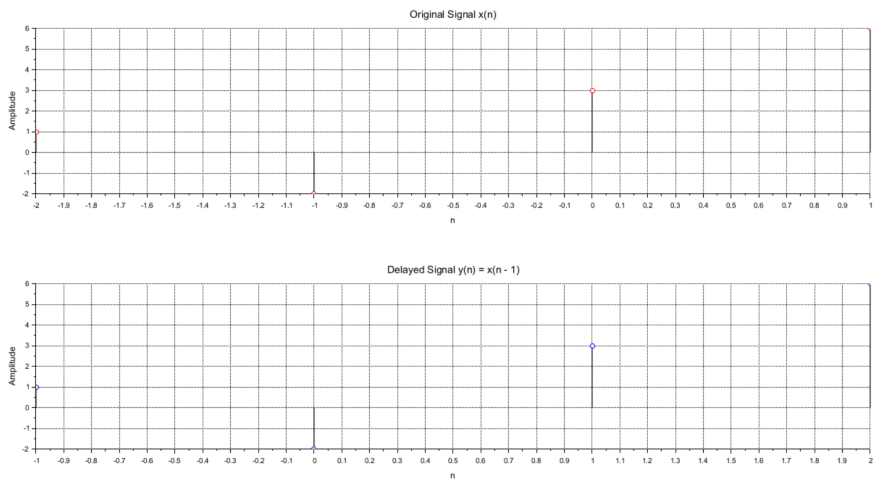

Figure 1: *Exercise 1 Output*

This is the source code for Exercise 2:

```
function [yn, yorigin] = advance(xn, xorigin, k)
    // 1. Calculate the new origin position
    // Shifting x(n+k) moves the origin index back by k
    yorigin = xorigin + k;
    yn = xn; // The values in the vector remain the same

    // 2. Determine time axes for plotting
    // Signal x starts at 1-xorigin and ends at length(xn)-xorigin
    nx = (1:length(xn)) - xorigin;
    ny = (1:length(yn)) - yorigin;

    // 3. Graphical Display
    clf();
    // Plot Original x(n)
    subplot(2,1,1);
    plot2d3('gnn', nx, xn);
    plot(nx, xn, 'ro');
    xtitle( Original Signal x(n) ,  n ,  Amplitude );
    xgrid();

    // Plot Delayed y(n)
    subplot(2,1,2);
    plot2d3('gnn', ny, yn);
    plot(ny, yn, 'bo');
    xtitle( Advanced Signal y(n) = x(n +  +string(k)+ ) ,  n ,  Amplitude );
    xgrid();

    printf( Original Origin: %d\n , xorigin);
    printf( New Origin (yorigin): %d\n , yorigin);
endfunction
```

To advance the signal by k, the function moves the origin index forward by adding k to `xorigin`. Like the delay operation, the signal array itself remains identical. The time axes are derived using the signal length and the respective origin positions, and the signals are displayed using Scilab's plotting tools.

This is the output for Exercise 2:

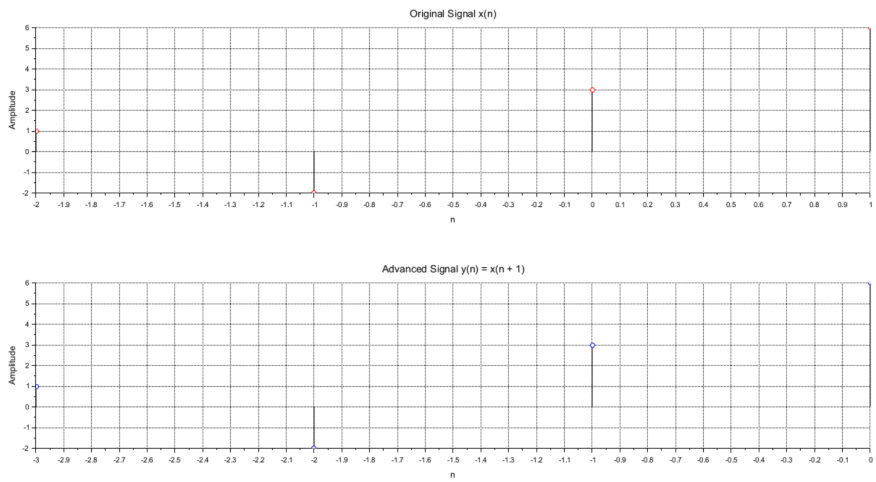

Figure 2: *Output for Exercise 2*

1.4.3 Exercise 3

`function [yn, yorigin] = fold (xn, xorigin)` performs folding operation $y(n) = x(-n)$, where:

- The discrete-time signal $x(n)$ is presented by vector `xn`.
- `xorigin` indicates the position of origin in vector $x(n)$.
- `yn` is vector of the output signal.
- `yorigin` indicates the position of origin in vector $y(n)$.
- $x(n)$ and $y(n)$ are graphically displayed in the same figure.

Solution

This is the source code for Exercise 3:

```
function [yn, yorigin] = fold(xn, xorigin)
    // 1. Calculate the new origin position
    yorigin = length(xn) - xorigin + 1;
    yn = xn($:-1:1); // The values in the vector remain the same

    // 2. Determine time axes for plotting
    // Signal x starts at 1-xorigin and ends at length(xn)-xorigin
    nx = (1:length(xn)) - xorigin;
```

```
    ny = (1:length(yn)) - yorigin;


    // 3. Graphical Display
    clf();
    // Plot Original x(n)
    subplot(2,1,1);
    plot2d3('gnn', nx, xn);
    plot(nx, xn, 'ro');
    xtitle( Original Signal x(n) ,  n ,  Amplitude );
    xgrid();

    // Plot Delayed y(n)
    subplot(2,1,2);
    plot2d3('gnn', ny, yn);
    plot(ny, yn, 'bo');
    xtitle( Fold Signal y(n) = x(-n) ,  n ,  Amplitude );
    xgrid();

    printf( Original Origin: %d\n , xorigin);
    printf( New Origin (yorigin): %d\n , yorigin);
endfunction
```

The folding operation physically reverses the input array using Scilab's array reversal syntax `xn($:-1:1)`. Because the array is flipped, the new origin index must be recomputed as `length(xn) - xorigin + 1` to maintain correct time alignment. Time ranges are established, and both graphs are plotted sequentially.

This is the output for Exercise 3:

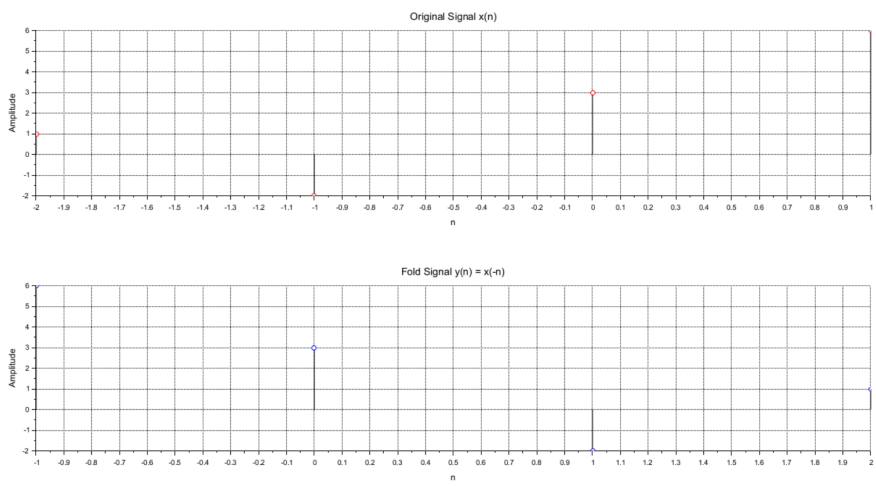

Figure 3: *Output for Exercise 3*

1.4.4 Exercise 4

`function [yn, yorigin] = add (x1n, x1origin, x2n, x2origin)` performs addition operation $y(n) = x_1(n) + x_2(n)$, where:

- The discrete-time signal $x_1(n)$ and $x_2(n)$ are presented by vectors `x1n` and `x2n`, respectively.
- `x1origin` and `x2origin` indicate the origin's position of signal $x_1(n)$ and $x_2(n)$, respectively.
- `yn` is vector of the output signal.
- `yorigin` indicates the origin's position of signal $y(n)$.
- $x_1(n)$, $x_2(n)$ and $y(n)$ are graphically displayed in the same figure.

For example: To determine $y(n) = x_1(n) + x_2(n)$, where $x_1(n) = \{\underset{\uparrow}{0}, 1, 3, -2\}$ and $x_2(n) = \{1, \underset{\uparrow}{1}, 2, 3\}$. You can use your own function by calling `[yn, yorigin] = add ([0, 1, 3, -2], 1, [1, 1, 2, 3], 2)`. Then, the output will be:

```
yn = [1,  1, 3,  6,  -2 ]
yorigin = 1
```

Additionally, you have to graphically display $x_1(n)$, $x_2(n)$ and $y(n)$ in the same figure.

Solution

This is the source code for Exercise 4:

```
function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
    // 1. Determine the time boundaries for both signals
    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;

    // 2. Define the universal time range
    n_min = min(min(n1), min(n2));
    n_max = max(max(n1), max(n2));
    ny = n_min : n_max;

    // 3. Initialize padded signals with zeros
    y1 = zeros(1, length(ny));
    y2 = zeros(1, length(ny));

    // 4. Insert original signals into the padded vectors
    // Find the starting position in the 'ny' vector for each signal
    pos1 = find(ny == min(n1));
    y1(pos1 : pos1 + length(x1n) - 1) = x1n;

    pos2 = find(ny == min(n2));
    y2(pos2 : pos2 + length(x2n) - 1) = x2n;

    // 5. Addition
    yn = y1 + y2;
    yorigin = find(ny == 0); // Find where n=0 is in the new vector
```

```
    // 6. Graphical Display
    clf();
    subplot(3,1,1);
    plot2d3('gnn', ny, y1); plot(ny, y1, 'ro');
    xtitle( Signal x1(n) ,  n ,  Amp ); xgrid();

    subplot(3,1,2);
    plot2d3('gnn', ny, y2); plot(ny, y2, 'bo');
    xtitle( Signal x2(n) ,  n ,  Amp ); xgrid();

    subplot(3,1,3);
    plot2d3('gnn', ny, yn); plot(ny, yn, 'go');
    xtitle( Result y(n) = x1 + x2 ,  n ,  Amp ); xgrid();
endfunction
```

Adding two signals requires aligning them on a common time axis. The script calculates universal time boundaries (`n_min` and `n_max`) based on the extents of both inputs. It creates two zero-padded arrays, `y1` and `y2`, over this combined range and maps the original values into their correct positions using the `find` function. Once aligned, they are added together (`yn = y1 + y2`), and the new origin is identified where the time index equals zero.

This is the output for Exercise 4:

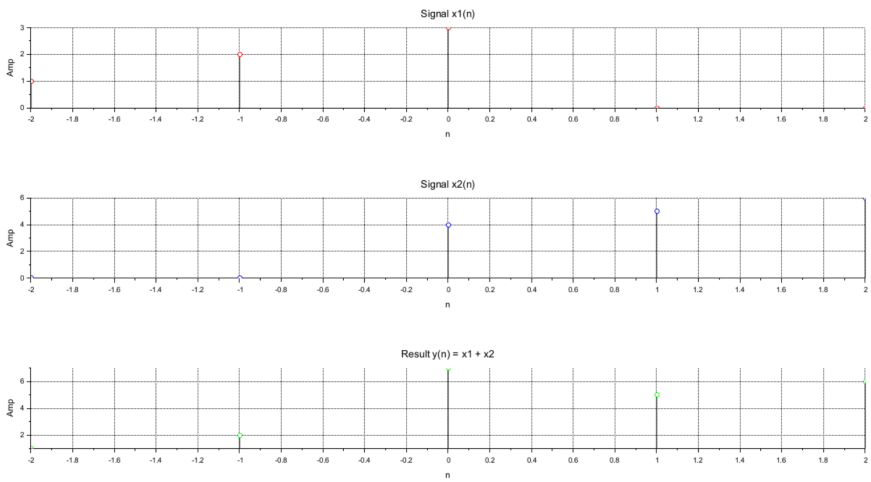

Figure 4: *Output for Exercise 4*

1.4.5 Exercise 5

`function [yn, yorigin] = multi (x1n, x1origin, x2n, x2origin)` performs multiplication operation $y(n) = x_1(n) \cdot x_2(n)$, where:

- The discrete-time signal $x_1(n)$ and $x_2(n)$ are presented by vectors `x1n` and `x2n`, respectively.

- `x1origin` and `x2origin` indicate the origin's position of signal $x_1(n)$ and $x_2(n)$, respectively.
- `yn` is vector of the output signal.
- `yorigin` indicates the origin's position of signal $y(n)$.
- $x_1(n)$, $x_2(n)$ and $y(n)$ are graphically displayed in the same figure.

Solution

This is the source code for Exercise 5:

```
function [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)
    // 1. Determine the time boundaries for both signals
    n1 = (1:length(x1n)) - x1origin;
    n2 = (1:length(x2n)) - x2origin;

    // 2. Define the universal time range
    n_min = min(min(n1), min(n2));
    n_max = max(max(n1), max(n2));
    ny = n_min : n_max;

    // 3. Initialize padded signals with zeros
    y1 = zeros(1, length(ny));
    y2 = zeros(1, length(ny));

    // 4. Insert original signals into the padded vectors
    // Find the starting position in the 'ny' vector for each signal
    pos1 = find(ny == min(n1));
    y1(pos1 : pos1 + length(x1n) - 1) = x1n;

    pos2 = find(ny == min(n2));
    y2(pos2 : pos2 + length(x2n) - 1) = x2n;

    // 5. Addition
    yn = y1 .* y2;
    yorigin = find(ny == 0); // Find where n=0 is in the new vector

    // 6. Graphical Display
    clf();
    subplot(3,1,1);
    plot2d3('gnn', ny, y1); plot(ny, y1, 'ro');
    xtitle( Signal x1(n) ,  n ,  Amp ); xgrid();

    subplot(3,1,2);
    plot2d3('gnn', ny, y2); plot(ny, y2, 'bo');
    xtitle( Signal x2(n) ,  n ,  Amp ); xgrid();

    subplot(3,1,3);
    plot2d3('gnn', ny, yn); plot(ny, yn, 'go');
    xtitle( Result y(n) = x1.x2 ,  n ,  Amp ); xgrid();
endfunction
```

```
[yn, yorigin] = multi([1, 2, 3, 4], 2, [3, 4, 5, 6], 3);
```

This is the output for Exercise 5:

Similar to addition, the signals must first be mapped to a universal time range spanning from the minimum to the maximum time indices of both signals. The code initializes zero-padded arrays and inserts the original signal values at their respective starting positions. The output is produced by element-wise multiplication, and the new origin is located at the zero time index.

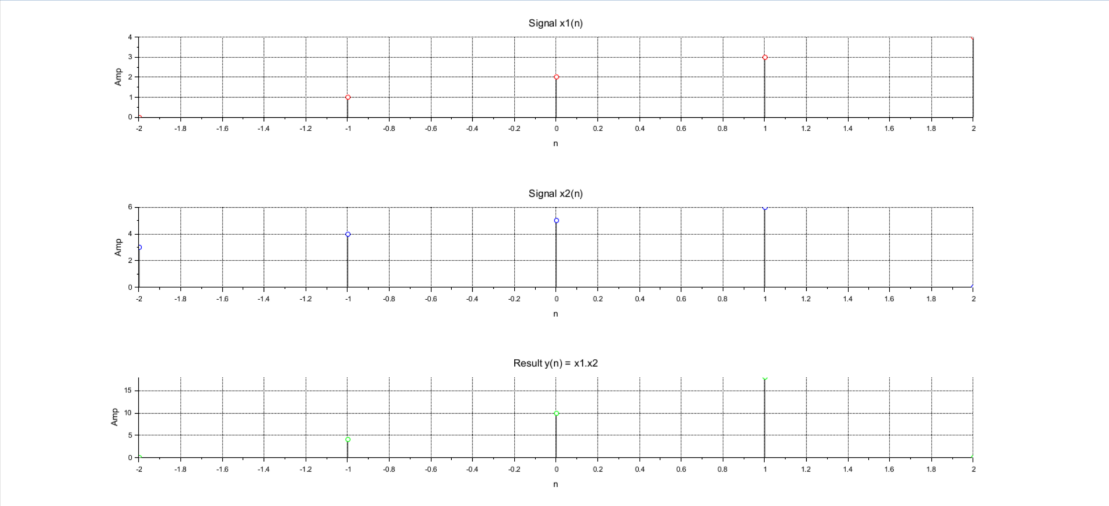

Figure 5: *Output for Exercise 5*

1.4.6 Exercise 6

`function [yn, yorigin] = convolution (xn, xorigin, hn, horigin)` performs convolution $y(n) = x(n) * h(n)$, where:

- $x(n)$ is the input signal and $h(n)$ is system characteristic's function.
- `xorigin` and `horigin` indicates the origin's position of $x(n)$ and $h(n)$, respectively.
- `yn` is vector of the output signal.
- `yorigin` indicates the origin's position of signal $y(n)$.

Solution

This is the source code for Exercise 6:

```
function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)
    // 1. Perform the mathematical convolution
    // convol() handles the sum of products for us
    yn = convol(xn, hn);

    // 2. Calculate the new origin position
    // Formula: Index of y(0) = Index of x(0) + Index of h(0) - 1
    yorigin = xorigin + horigin - 1;
```

```
    // 3. Define time axes for plotting
    nx = (1:length(xn)) - xorigin;
    nh = (1:length(hn)) - horigin;
    ny = (1:length(yn)) - yorigin;

    // 4. Graphical Display
    clf();

    // Plot Input x(n)
    subplot(3,1,1);
    plot2d3('gnn', nx, xn); plot(nx, xn, 'ro');
    xtitle( Input Signal x(n) ,  n ,  Amp ); xgrid();

    // Plot Impulse Response h(n)
    subplot(3,1,2);
    plot2d3('gnn', nh, hn); plot(nh, hn, 'bo');
    xtitle( Impulse Response h(n) ,  n ,  Amp ); xgrid();

    // Plot Result y(n)
    subplot(3,1,3);
    plot2d3('gnn', ny, yn); plot(ny, yn, 'go');
    xtitle( Convolution Result y(n) = x(n)*h(n) ,  n ,  Amp ); xgrid();

    printf( Resulting Origin Position (yorigin): %d\n , yorigin);
endfunction
```

The script handles the mathematical sum of products using Scilab's built-in `convol(xn, hn)` function[cite: 432, 434]. The fundamental property of discrete convolution dictates that the new origin index is computed as `xorigin + horigin - 1`. Time axes are formulated for the input, impulse response, and output, and all three are plotted in a 3-row subplot for clear comparison.

This is the output for Exercise 6:

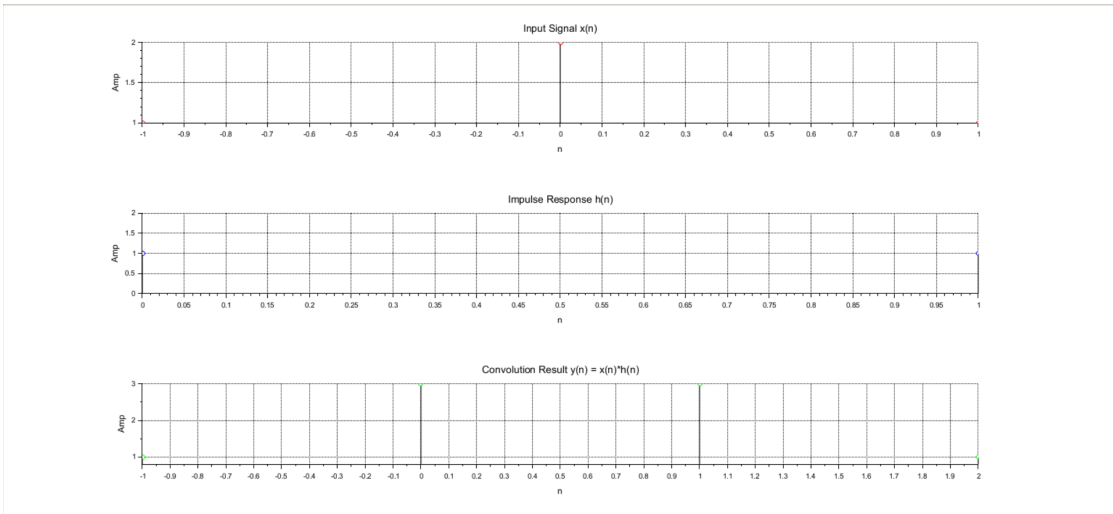

Figure 6: *Output for Exercise 6*

1.5 Additional Exercise

Complete exercises 2.1, 2.2, 2.4 and 2.6 in the reference document (both theoretical and practical parts on Scilab) in the attachment.

1.5.1 Exercise 2.1

A discrete-time signal $x(n)$ is defined as

$$x(n) = \begin{cases} 1 + \frac{n}{3}, & -3 \leq n \leq -1 \\ 1, & 0 \leq n \leq 3 \\ 0, & \text{elsewhere} \end{cases}$$

(a) Determine its values and sketch the signal $x(n)$.

(b) Sketch the signals that result if we:

1. First fold $x(n)$ and then delay the resulting signal by four samples.

2. First delay $x(n)$ by four samples and then fold the resulting signal.

(c) Sketch the signal $x(-n+4)$.

(d) Compare the results in parts (b) and (c) and derive a rule for obtaining the signal $x(-n+k)$ from $x(n)$.

(e) Can you express the signal $x(n)$ in terms of signals $\delta(n)$ and $u(n)$?

Solution

(a) Values and sketch of $x(n)$:

Evaluating the piecewise function for integer values of n:

- For $-3 \leq n \leq -1$, using $x(n) = 1 + \frac{n}{3}$:

$$x(-3) = 0, \quad x(-2) = \frac{1}{3}, \quad x(-1) = \frac{2}{3}$$

- For $0 \leq n \leq 3$, using $x(n) = 1$:

$$x(0) = 1, \quad x(1) = 1, \quad x(2) = 1, \quad x(3) = 1$$

- Elsewhere: $x(n) = 0$

Represented as a sequence (the arrow indicates the origin $n = 0$):

$$x(n) = \left\{0, \frac{1}{3}, \frac{2}{3}, \underset{\uparrow}{1}, 1, 1, 1\right\}$$

(b) Operations on $x(n)$:

1. **Fold then delay by 4:** Let the folded signal be $v_1(n) = x(-n)$. Delaying $v_1(n)$ by 4 samples yields:

$$y_1(n) = v_1(n-4) = x(-(n-4)) = x(-n+4)$$

2. **Delay by 4 then fold:** Let the delayed signal be $v_2(n) = x(n-4)$. Folding $v_2(n)$ yields:

$$y_2(n) = v_2(-n) = x(-n-4)$$

(c) Signal $x(-n+4)$: To find the non-zero range, we set the bounds $-3 \leq -n+4 \leq 3$, which simplifies to $1 \leq n \leq 7$. Reversing the original sequence and shifting right by 4 gives:

$$x(-n+4) = \{0, \underset{\uparrow}{0}, 1, 1, 1, 1, \frac{2}{3}, \frac{1}{3}, 0\}$$

(d) Comparison and Rule: The signal $x(-n+4)$ from part (c) matches the mathematical result of part (b) 1.

Rule: To obtain the signal $x(-n+k)$ from $x(n)$ for $k > 0$, we must **first fold** $x(n)$ to get $x(-n)$, and **then delay** the folded signal by k samples (shift to the right by k).

(e) Expression in terms of $\delta(n)$ and $u(n)$: The signal consists of individual fractional values at $n = -2$ and $n = -1$, followed by a sequence of 1s from $n = 0$ to $n = 3$. This can be constructed using unit impulses for the discrete points and a rectangular pulse (difference of two unit steps) for the flat top:

$$x(n) = \frac{1}{3}\delta(n+2) + \frac{2}{3}\delta(n+1) + u(n) - u(n-4)$$

Sketching for (a), (b) and (c) in Scilab:

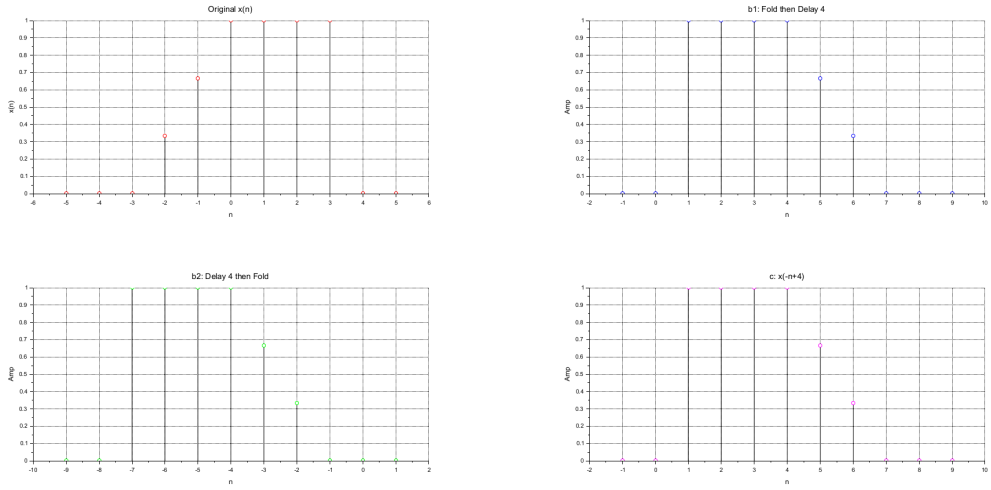

Figure 7: *Output for Exercise 2.1*

This is the code for sketching Exercise 2.1:

```
// --- Problem 2.1 ---
scf(1); // Open a second window so they don't overlap
n = -5:5; // Use a wide range to see the  elsewhere  zeros

// Define original x(n) using logical segments
part1 = (1 + n/3) .* bool2s(n >= -3 & n <= -1);
part2 = (1) .* bool2s(n >= 0 & n <= 3);
x_p = part1 + part2;

// (b) 1. Fold then delay 4: x(-n) then x(-(n-4)) = x(-n+4)
n_fold_delay = -(n - 4);
y_b1 = x_p;

// (b) 2. Delay 4 then fold: x(n-4) then x(-n-4)
n_delay_fold = (-n) - 4;
y_b2 = x_p;

// (c) y = x(-n + 4) -> This is actually the same as (b1)
n_c = -n + 4;
y_c = x_p;

// Plotting Problem 2.1
subplot(2,2,1); plot2d3('gnn', n, x_p); plot(n, x_p, 'ro');
title( Original x(n) ); xlabel( n ); ylabel( x(n) ); xgrid();

subplot(2,2,2); plot2d3('gnn', n_fold_delay, y_b1); plot(n_fold_delay, y_b1, 'bo')
    ;
title( b1: Fold then Delay 4 ); xlabel( n ); ylabel( Amp ); xgrid();

subplot(2,2,3); plot2d3('gnn', n_delay_fold, y_b2); plot(n_delay_fold, y_b2, 'go')
```

```
    ;
title( b2: Delay 4 then Fold ); xlabel( n ); ylabel( Amp ); xgrid();

subplot(2,2,4); plot2d3('gnn', n_c, y_c); plot(n_c, y_c, 'mo');
title( c: x(-n+4) ); xlabel( n ); ylabel( Amp ); xgrid();
```

1.5.2 Exercise 2.2

A discrete-time signal $x(n)$ is shown in Fig. P2.2. Sketch and label carefully each of the following signals.

(a) $x(n-2)$

(b) $x(4-n)$

(c) $x(n+2)$

(d) $x(n)u(2-n)$

(e) $x(n-1)\delta(n-3)$

(f) $x(n^2)$

(g) even part of $x(n)$

(h) odd part of $x(n)$

Solution

First, we define the base discrete-time signal $x(n)$ extracted from the figure. Writing this as a sequence (where the arrow indicates the origin at $n = 0$):

$$x(n) = \left\{1, \underset{\uparrow}{1}, 1, 1, \frac{1}{2}, \frac{1}{2}\right\}$$

Specifically: $x(-1) = 1$, $x(0) = 1$, $x(1) = 1$, $x(2) = 1$, $x(3) = \frac{1}{2}$, $x(4) = \frac{1}{2}$, *and* $x(n) = 0$ *elsewhere.*

(a) $y(n) = x(n-2)$
This is a right shift by 2 samples. The non-zero values shift from $n \in [-1, 4]$ to $n \in [1, 6]$.

$$y(n) = \left\{\underset{\uparrow}{0}, 1, 1, 1, 1, \frac{1}{2}, \frac{1}{2}\right\}$$

(b) $y(n) = x(4-n) = x(-(n-4))$
This requires folding the signal and shifting it right by 4. For instance, $y(0) = x(4) = \frac{1}{2}$ and $y(5) = x(-1) = 1$.

$$y(n) = \left\{\underset{\uparrow}{\frac{1}{2}}, \frac{1}{2}, 1, 1, 1, 1\right\}$$

(c) $y(n) = x(n+2)$

This is a left shift by 2 samples. The non-zero values shift from $n \in [-1, 4]$ to $n \in [-3, 2]$.

$$y(n) = \left\{1, 1, 1, \underset{\uparrow}{1}, \frac{1}{2}, \frac{1}{2}\right\}$$

(d) $y(n) = x(n)u(2-n)$
The step function $u(2-n)$ is 1 for $n \leq 2$ and 0 for $n > 2$. This truncates the signal, keeping only the values from $n = -1$ to $n = 2$.

$$y(n) = \left\{1, \underset{\uparrow}{1}, 1, 1\right\}$$

(e) $y(n) = x(n-1)\delta(n-3)$
The unit impulse $\delta(n-3)$ is 1 strictly at $n = 3$ and 0 elsewhere. Thus, we only evaluate $x(n-1)$ at $n = 3$, which gives $x(2) \cdot 1 = 1 \cdot 1 = 1$. The signal is only non-zero at $n = 3$.

$$y(n) = \left\{\underset{\uparrow}{0}, 0, 0, 1\right\}$$

(f) $y(n) = x(n^2)$
We map the values for integer squares: $y(0) = x(0) = 1$, $y(\pm 1) = x(1) = 1$, and $y(\pm 2) = x(4) = \frac{1}{2}$. All other integer values of n yield an index greater than 4, where x is 0.

$$y(n) = \left\{\frac{1}{2}, 1, \underset{\uparrow}{1}, 1, \frac{1}{2}\right\}$$

(g) **Even part:** $x_e(n) = \frac{1}{2}[x(n) + x(-n)]$
Calculating for each index symmetrically around the origin out to $n = \pm 4$:

$$x_e(n) = \left\{\frac{1}{4}, \frac{1}{4}, \frac{1}{2}, 1, \underset{\uparrow}{1}, 1, \frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right\}$$

(h) **Odd part:** $x_o(n) = \frac{1}{2}[x(n) - x(-n)]$
Calculating for each index symmetrically around the origin (note that the origin $n = 0$ is always 0 for odd signals):

$$x_o(n) = \left\{-\frac{1}{4}, -\frac{1}{4}, -\frac{1}{2}, 0, \underset{\uparrow}{0}, 0, \frac{1}{2}, \frac{1}{4}, \frac{1}{4}\right\}$$

This is the sketching for problem (a), (b), (c), (d) for Exercise 2.2:

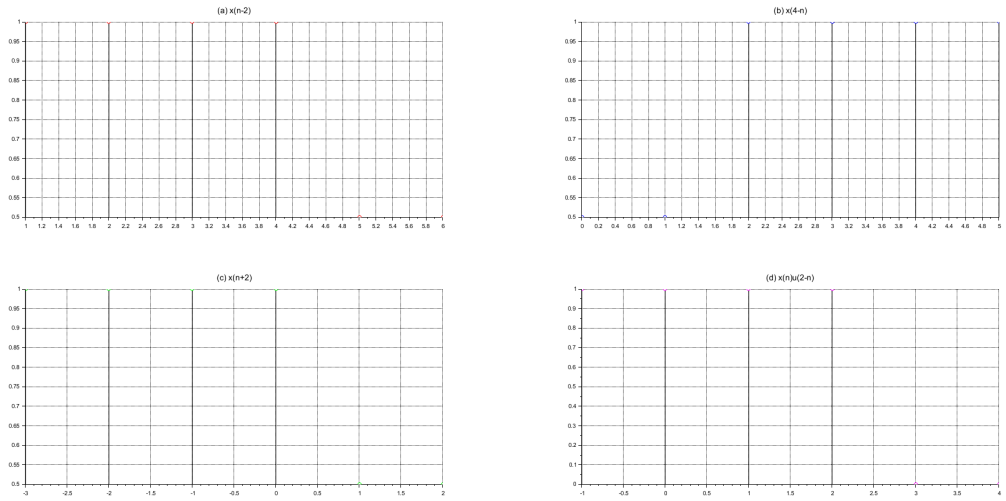

Figure 8: *Output for problem (a), (b), (c), (d) in Exercise 2.2*

This is the sketching for problem (a), (b), (c), (d) for Exercise 2.2:

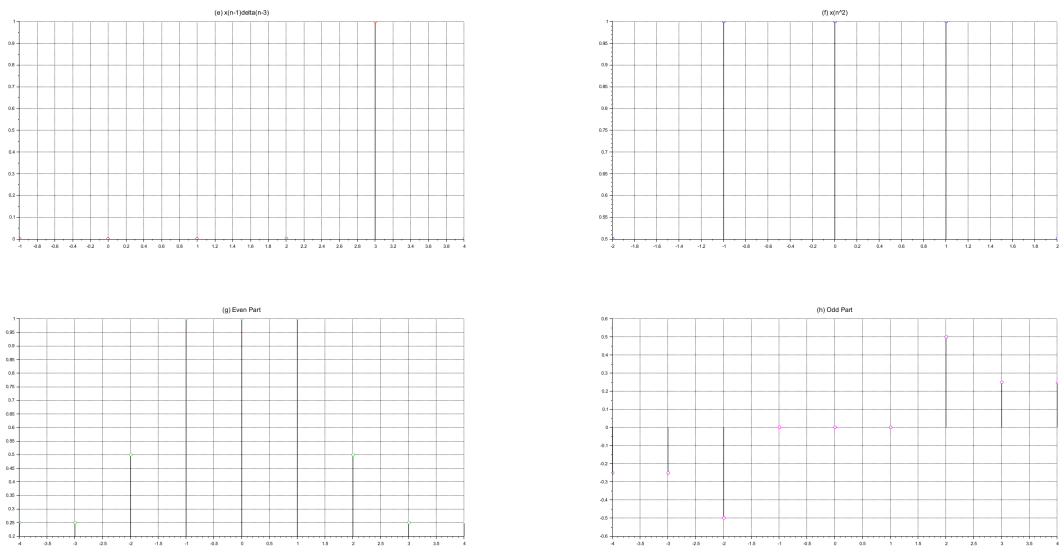

Figure 9: *Output for problem (e), (f), (g), (h) in Exercise 2.2*

This is the code for generating sketch for Exercise 2.2:

```
// --- 1. Define Original Signal ---
x = [1, 1, 1, 1, 0.5, 0.5];
x_orig = 2; // Index 2 is n=0
n = (1:length(x)) - x_orig;

// --- 2. Window 1 Logic ---
```

```
na = n + 2;         // x(n-2)
nb = -(n - 4);      // x(4-n)
nc = n - 2;         // x(n+2)
yd = x .* bool2s(n <= 2); // x(n)u(2-n)

scf(0); // Open Window 0
clf();
subplot(2,2,1); plot2d3('gnn', na, x); plot(na, x, 'ro'); title( (a) x(n-2) );
    xgrid();
subplot(2,2,2); plot2d3('gnn', nb, x); plot(nb, x, 'bo'); title( (b) x(4-n) );
    xgrid();
subplot(2,2,3); plot2d3('gnn', nc, x); plot(nc, x, 'go'); title( (c) x(n+2) );
    xgrid();
subplot(2,2,4); plot2d3('gnn', n, yd); plot(n, yd, 'mo'); title( (d) x(n)u(2-n) );
     xgrid();

// --- 3. Window 2 Logic ---

// (e) x(n-1)delta(n-3)
ye = zeros(1, length(n));
idx_3 = find(n == 3);
ye(idx_3) = x(find(n == 2));

// (f) x(n^2)
nf = -2:2;
yf = zeros(1, length(nf));
for i = 1:length(nf)
    n_sq = nf(i)^2;
    idx = find(n == n_sq);
    yf(i) = x(idx);
end

// (g & h) Even and Odd
n_sym = -4:4;
x_pad = [0, 0, 0, 1, 1, 1, 1, 0.5, 0.5]; // Padded n = -4:4
x_fold = x_pad($:-1:1);
xe = 0.5 * (x_pad + x_fold);
xo = 0.5 * (x_pad - x_fold);

scf(1); // Open Window 1
clf();
subplot(2,2,1); plot2d3('gnn', n, ye); plot(n, ye, 'ro'); title( (e) x(n-1)delta(n
    -3) ); xgrid();
subplot(2,2,2); plot2d3('gnn', nf, yf); plot(nf, yf, 'bo'); title( (f) x(n^2) );
    xgrid();
subplot(2,2,3); plot2d3('gnn', n_sym, xe); plot(n_sym, xe, 'go'); title( (g) Even
    Part ); xgrid();
subplot(2,2,4); plot2d3('gnn', n_sym, xo); plot(n_sym, xo, 'mo'); title( (h) Odd
    Part ); xgrid();
```

1.5.3 Exercise 2.4

Show that any signal can be decomposed into an even and an odd component. Is the decomposition unique? Illustrate your arguments using the signal

$$x(n) = \{2, 3, \underset{\uparrow}{4}, 5, 6\}$$

Solution

1. Proof of Decomposition:
Let $x(n)$ be any arbitrary discrete-time signal. We define two new signals:

$$x_e(n) = \frac{1}{2}[x(n) + x(-n)]$$
$$x_o(n) = \frac{1}{2}[x(n) - x(-n)]$$

First, we verify their symmetry properties. For $x_e(n)$:

$$x_e(-n) = \frac{1}{2}[x(-n) + x(-(-n))] = \frac{1}{2}[x(-n) + x(n)] = x_e(n)$$

Thus, $x_e(n)$ is an even signal. For $x_o(n)$:

$$x_o(-n) = \frac{1}{2}[x(-n) - x(-(-n))] = \frac{1}{2}[x(-n) - x(n)] = -x_o(n)$$

Thus, $x_o(n)$ is an odd signal. Adding these two components yields:

$$x_e(n) + x_o(n) = \frac{1}{2}x(n) + \frac{1}{2}x(-n) + \frac{1}{2}x(n) - \frac{1}{2}x(-n) = x(n)$$

This proves that any signal can be decomposed into an even and an odd component.

2. Proof of Uniqueness:
Assume there exists another decomposition $x(n) = y_e(n) + y_o(n)$, where $y_e(n)$ is even and $y_o(n)$ is odd. Evaluating $x(-n)$:

$$x(-n) = y_e(-n) + y_o(-n) = y_e(n) - y_o(n)$$

Adding $x(n)$ and $x(-n)$:

$$x(n) + x(-n) = [y_e(n) + y_o(n)] + [y_e(n) - y_o(n)] = 2y_e(n) \implies y_e(n) = \frac{1}{2}[x(n) + x(-n)] = x_e(n)$$

Subtracting $x(-n)$ from $x(n)$:

$$x(n) - x(-n) = [y_e(n) + y_o(n)] - [y_e(n) - y_o(n)] = 2y_o(n) \implies y_o(n) = \frac{1}{2}[x(n) - x(-n)] = x_o(n)$$

Since $y_e(n) = x_e(n)$ and $y_o(n) = x_o(n)$, the decomposition is strictly unique.

3. Illustration:

Given the signal $x(n) = \{2, 3, \underset{\uparrow}{4}, 5, 6\}$, we can write its folded version as:

$$x(-n) = \{6, 5, \underset{\uparrow}{4}, 3, 2\}$$

Calculating the even component $x_e(n) = \frac{1}{2}[x(n) + x(-n)]$:

$$x_e(n) = \frac{1}{2}\{8, 8, \underset{\uparrow}{8}, 8, 8\} = \{4, 4, \underset{\uparrow}{4}, 4, 4\}$$

Calculating the odd component $x_o(n) = \frac{1}{2}[x(n) - x(-n)]$:

$$x_o(n) = \frac{1}{2}\{-4, -2, \underset{\uparrow}{0}, 2, 4\} = \{-2, -1, \underset{\uparrow}{0}, 1, 2\}$$

Verifying the sum $x_e(n) + x_o(n)$:

$$x_e(n) + x_o(n) = \{2, 3, \underset{\uparrow}{4}, 5, 6\} = x(n)$$

This matches the original signal, verifying the decomposition.

1.5.4 Exercise 2.6

Consider the system

$$y(n) = \mathcal{T}[x(n)] = x(n^2)$$

(a) Determine if the system is time invariant.

(b) To clarify the result in part (a) assume that the signal

$$x(n) = \begin{cases} 1, & 0 \le n \le 3 \\ 0, & \text{elsewhere} \end{cases}$$

is applied into the system.

(1) Sketch the signal $x(n)$.

(2) Determine and sketch the signal $y(n) = \mathcal{T}[x(n)]$.

(3) Sketch the signal $y_2'(n) = y(n-2)$.

(4) Determine and sketch the signal $x_2(n) = x(n-2)$.

(5) Determine and sketch the signal $y_2(n) = \mathcal{T}[x_2(n)]$.

(6) Compare the signals $y_2(n)$ and $y(n-2)$. What is your conclusion?

(c) Repeat part (b) for the system

$$y(n) = x(n) - x(n-1)$$

Can you use this result to make any statement about the time invariance of this system? Why?

(d) Repeat parts (b) and (c) for the system

$$y(n) = \mathcal{T}[x(n)] = nx(n)$$

Solution

(a) Time Invariance of $y(n) = x(n^2)$: Let $y(n,k)$ be the response of the system to a delayed input $x(n-k)$.

$$y(n,k) = \mathcal{T}[x(n-k)] = x(n^2 - k)$$

Now, we delay the output $y(n)$ by k samples:

$$y(n-k) = x((n-k)^2) = x(n^2 - 2nk + k^2)$$

Since $y(n,k) \neq y(n-k)$ for $k \neq 0$, the system is **time variant**.

(b) Example for $y(n) = x(n^2)$:

(1) $x(n) = \{\underset{\uparrow}{1}, 1, 1, 1\}$

(2) $y(n) = x(n^2)$. Evaluating for integer n: At $n = 0, n^2 = 0 \implies y(0) = x(0) = 1$ At $n = \pm 1, n^2 = 1 \implies y(\pm 1) = x(1) = 1$ At $n = \pm 2, n^2 = 4 \implies y(\pm 2) = x(4) = 0$

$$y(n) = \{1, \underset{\uparrow}{1}, 1\}$$

(3) $y_2'(n) = y(n-2)$. Shifting $y(n)$ to the right by 2:

$$y_2'(n) = \{\underset{\uparrow}{0}, 1, 1, 1\}$$

(4) $x_2(n) = x(n-2)$. Shifting $x(n)$ to the right by 2:

$$x_2(n) = \{\underset{\uparrow}{0}, 0, 1, 1, 1, 1\}$$

(5) $y_2(n) = \mathcal{T}[x_2(n)] = x_2(n^2) = x(n^2 - 2)$. $x(m)$ is only non-zero for $m \in \{0, 1, 2, 3\}$. So we need $0 \leq n^2 - 2 \leq 3$, which means $2 \leq n^2 \leq 5$. The only integer n satisfying this is $n = \pm 2$.

At $n = \pm 2, y_2(\pm 2) = x(4-2) = x(2) = 1$.

$$y_2(n) = \{1, 0, \underset{\uparrow}{0}, 0, 1\}$$

(6) $y_2(n) \neq y_2'(n)$. This confirms the conclusion from part (a) that the system is **time variant**.

(c) System $y(n) = x(n) - x(n-1)$:

(1) $x(n) = \{\underset{\uparrow}{1}, 1, 1, 1\}$

(2) $y(n) = x(n) - x(n-1)$. $y(0) = 1 - 0 = 1$; $y(1) = 1 - 1 = 0$; $y(2) = y(3) = 0$; $y(4) = 0 - 1 = -1$.

$$y(n) = \{\underset{\uparrow}{1}, 0, 0, 0, -1\}$$

(3) $y_2'(n) = y(n-2) = \{\underset{\uparrow}{0}, 0, 1, 0, 0, 0, -1\}$

(4) $x_2(n) = x(n-2) = \{\underset{\uparrow}{0}, 0, 1, 1, 1, 1\}$

(5) $y_2(n) = x_2(n) - x_2(n-1)$. $y_2(2) = 1 - 0 = 1$; $y_2(3) = y_2(4) = y_2(5) = 0$; $y_2(6) = 0 - 1 = -1$.

$$y_2(n) = \{\underset{\uparrow}{0}, 0, 1, 0, 0, 0, -1\}$$

(6) $y_2(n) = y_2'(n)$.
Statement: While this specific input yields identical delayed outputs, we **cannot** definitively prove time invariance using just one test signal. To make a definitive statement, we must evaluate it analytically for an arbitrary $x(n)$: $\mathcal{T}[x(n-k)] = x(n-k) - x(n-k-1)$, which equals $y(n-k)$. Therefore, it is mathematically time invariant.

(d) System $y(n) = nx(n)$:

(1) $x(n) = \{\underset{\uparrow}{1}, 1, 1, 1\}$

(2) $y(n) = nx(n) = \{\underset{\uparrow}{0}, 1, 2, 3\}$

(3) $y_2'(n) = y(n-2) = \{\underset{\uparrow}{0}, 0, 0, 1, 2, 3\}$

(4) $x_2(n) = x(n-2) = \{\underset{\uparrow}{0}, 0, 1, 1, 1, 1\}$

(5) $y_2(n) = nx_2(n)$. $y_2(2) = 2(1) = 2$; $y_2(3) = 3(1) = 3$; $y_2(4) = 4$; $y_2(5) = 5$.

$$y_2(n) = \{\underset{\uparrow}{0}, 0, 2, 3, 4, 5\}$$

(6) $y_2(n) \neq y_2'(n)$. This confirms the system $y(n) = nx(n)$ is **time variant**.

This is the output of problem (b):

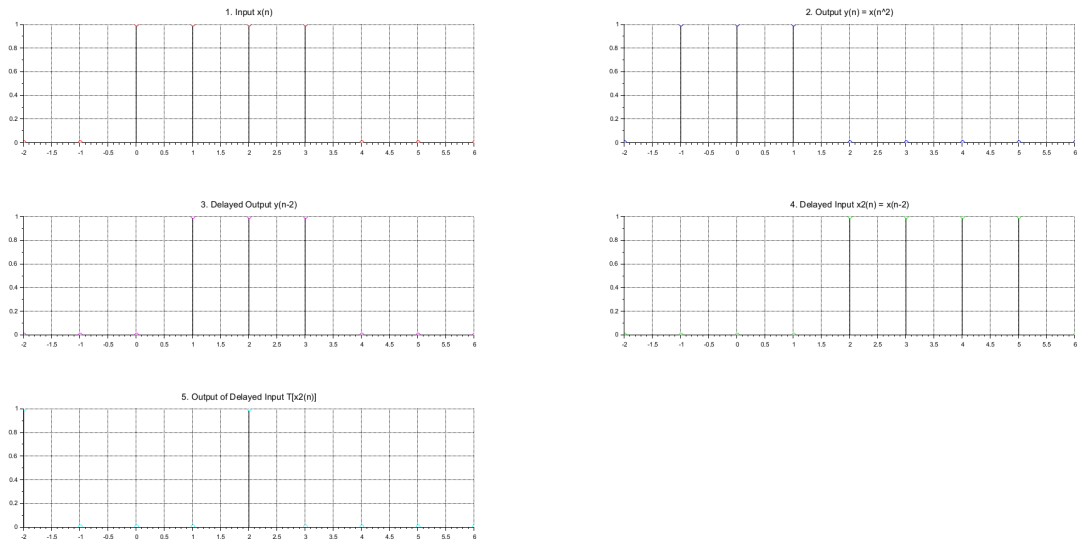

Figure 10: *Output for problem 2.6 (b)*

This is the output for problem (c):

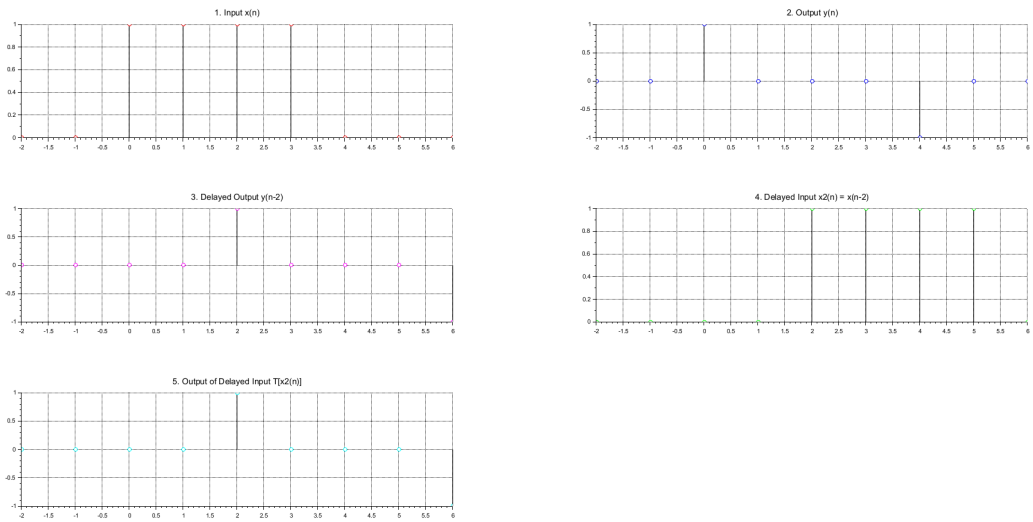

Figure 11: *Output for problem 2.6 (c)*

This is the output for problem (d):

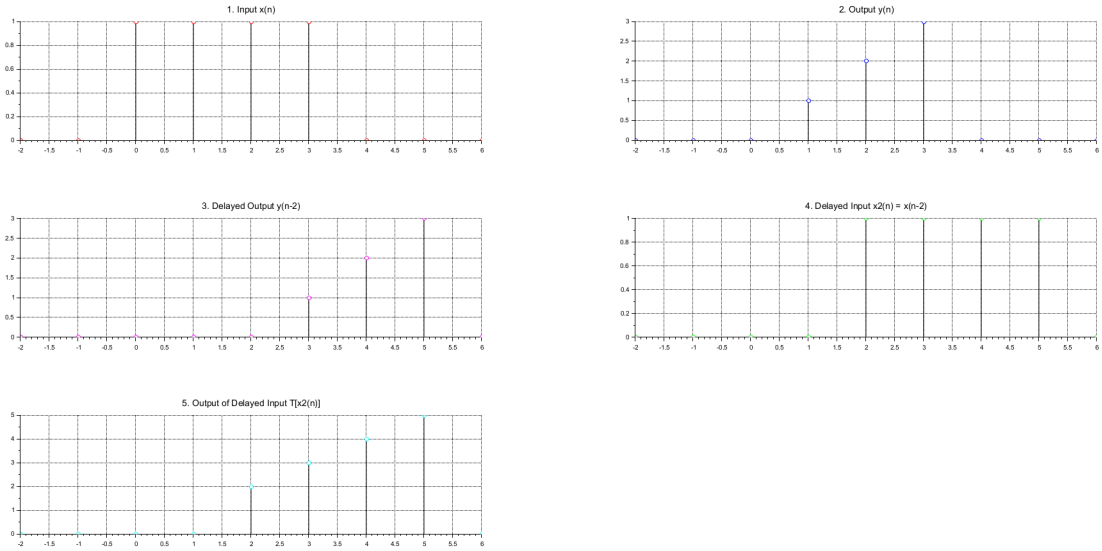

Figure 12: *Output for problem 2.6 (d)*

This is the Scilab code for generating sketch for Exercise 2.6:

```
// ==========================================
// PROBLEM 2.6: Time Invariance Tests
// ==========================================

// Define the piecewise input signal x(n) as a function
function val = x_in(idx)
    val = zeros(idx); // Default to 0
    val(idx >= 0 & idx <= 3) = 1; // Set to 1 in the range [0, 3]
endfunction

// Define a wide time axis for our sketches
n = -2:6;

// ------------------------------------------
// Part (b): y(n) = x(n^2)  [Time Variant]
// ------------------------------------------
scf(1); clf();
f = gcf(); f.figure_name =  Problem 2.6 (b): y(n) = x(n^2) ;
subplot(3,2,1); plot2d3('gnn', n, x_in(n)); plot(n, x_in(n), 'ro'); title( 1. Input x(n) ); xgrid();
subplot(3,2,2); plot2d3('gnn', n, x_in(n.^2)); plot(n, x_in(n.^2), 'bo'); title( 2. Output y(n) = x(n^2) ); xgrid();
subplot(3,2,3); plot2d3('gnn', n, x_in((n-2).^2)); plot(n, x_in((n-2).^2), 'mo'); title( 3. Delayed Output y(n-2) ); xgrid();
subplot(3,2,4); plot2d3('gnn', n, x_in(n-2)); plot(n, x_in(n-2), 'go'); title( 4. Delayed Input x2(n) = x(n-2) ); xgrid();
```

```
subplot(3,2,5); plot2d3('gnn', n, x_in(n.^2 - 2)); plot(n, x_in(n.^2 - 2), 'co'); title( 5. Output of Delayed Input T[x2(n)] ); xgrid();
// Compare Plot 3 and Plot 5: They are different -> Time Variant

// ----------------------------------------
// Part (c): y(n) = x(n) - x(n-1)  [Time Invariant]
// ----------------------------------------
scf(2); clf();
f = gcf(); f.figure_name =  Problem 2.6 (c): y(n) = x(n) - x(n-1) ;
y_c       = x_in(n) - x_in(n-1);
yd_c      = x_in(n-2) - x_in(n-3); // y(n-2)
y2_c      = x_in(n-2) - x_in(n-3); // T[x(n-2)]

subplot(3,2,1); plot2d3('gnn', n, x_in(n)); plot(n, x_in(n), 'ro'); title( 1. Input x(n) ); xgrid();
subplot(3,2,2); plot2d3('gnn', n, y_c); plot(n, y_c, 'bo'); title( 2. Output y(n) ); xgrid();
subplot(3,2,3); plot2d3('gnn', n, yd_c); plot(n, yd_c, 'mo'); title( 3. Delayed Output y(n-2) ); xgrid();
subplot(3,2,4); plot2d3('gnn', n, x_in(n-2)); plot(n, x_in(n-2), 'go'); title( 4. Delayed Input x2(n) = x(n-2) ); xgrid();
subplot(3,2,5); plot2d3('gnn', n, y2_c); plot(n, y2_c, 'co'); title( 5. Output of Delayed Input T[x2(n)] ); xgrid();
// Compare Plot 3 and Plot 5: They are exactly the same -> Time Invariant

// ----------------------------------------
// Part (d): y(n) = n*x(n)  [Time Variant]
// ----------------------------------------
scf(3); clf();
f = gcf(); f.figure_name =  Problem 2.6 (d): y(n) = n*x(n) ;
y_d       = n .* x_in(n);
yd_d      = (n-2) .* x_in(n-2); // y(n-2) replaces all 'n' with 'n-2'
y2_d      = n .* x_in(n-2);     // T[x(n-2)] only delays the signal, not the scaling factor 'n'

subplot(3,2,1); plot2d3('gnn', n, x_in(n)); plot(n, x_in(n), 'ro'); title( 1. Input x(n) ); xgrid();
subplot(3,2,2); plot2d3('gnn', n, y_d); plot(n, y_d, 'bo'); title( 2. Output y(n) ); xgrid();
subplot(3,2,3); plot2d3('gnn', n, yd_d); plot(n, yd_d, 'mo'); title( 3. Delayed Output y(n-2) ); xgrid();
subplot(3,2,4); plot2d3('gnn', n, x_in(n-2)); plot(n, x_in(n-2), 'go'); title( 4. Delayed Input x2(n) = x(n-2) ); xgrid();
subplot(3,2,5); plot2d3('gnn', n, y2_d); plot(n, y2_d, 'co'); title( 5. Output of Delayed Input T[x2(n)] ); xgrid();
// Compare Plot 3 and Plot 5: They are different -> Time Variant
```

2 Reference

The full source code and this report is available in the GitHub repository: https://github.com/l1ttled1no/DigitalSignalProcessing-Lab